

PROIECT BAZE DE DATE

**GESTIUNEA UNEI
PARTIDE DE MONOPOLY**

STUDENT: Soare Mihai

GRUPA: 152

Cuprins

1. Descrierea modelului real.....	3
2. Prezentarea constrângerilor.....	3
3. Proiectarea Diagramei Entitate-Relație.....	3
3.1 Descrierea entităților.....	3
3.2 Descrierea relațiilor.....	3
3.3 Descrierea atributelor.....	4
4. Transformarea Diagramei ER în Diagrama Conceptuală.....	4
5. Scheme relaționale.....	4
6. Crearea tabelelor în SQL și inserarea datelor.....	4
6.1 Crearea tabelelor.....	4
6.2 Inserarea datelor.....	4

1. Descrierea modelului real

Modelul gestionează desfășurarea și starea partidelor de Monopoly. Monopoly este un joc de societate în care mai mulți jucători se deplasează pe o tablă, cumpără proprietăți, plătesc și încasează sume de bani și fac schimburi între ei. Baza de date memorează atât configurația fixă a jocului (tabla, proprietățile, grupurile de culoare, pachetul de cărți), cât și starea dinamică a fiecărei partide (jucători, poziții, bani, proprietăți deținute, aruncări de zaruri, cărți trase).

Utilitate: sistemul permite reluarea, analiza și vizualizarea partidelor. Deținerile nu se memorează direct, ci se deduc dintr-un jurnal de tranzacții de tip append-only: proprietarul curent al unui obiect într-o partidă este destinatarul (to_pawn) al celei mai recente tranzacții pentru acel obiect în acea partidă.

Entitățile modelului sunt:

- SETS – grupurile de culoare ale proprietăților (ex. Maro, Roșu) și prețul unei case pe grupul respectiv.
- PROPERTIES – terenurile care pot fi cumpărate; fiecare aparține exact unui grup de culoare.
- BOARD_SQUARES – pozițiile fixe de pe tabla de joc; o poziție poate găzdui cel mult o proprietate, iar unele poziții sunt speciale (GO, Închisoare, Șansă, taxe etc.).
- GAMES – o partidă concretă, cu sămânța generatorului aleator și un nume prietenos.
- PLAYERS – un pion care participă la o partidă; este identificat prin partida căreia îi aparține împreună cu numărul pionului. Este o entitate dependentă de GAMES.
- ITEMS – supertipul a tot ceea ce poate fi deținut sau tranzacționat; coloana type discriminează între cele două subtipuri.
- PROPERTY_ITEMS – subtip al ITEMS: dreptul de proprietate asupra unei PROPERTIES.
- CASHBAG_ITEMS – subtip al ITEMS: o sumă de bani.
- TRANSACTIONS – jurnalul mutărilor unui obiect între pionii în cadrul unei partide; from_pawn poate fi NULL când mutarea provine de la bancă. Cele două tranzacții ale unui schimb partajează același trade_key.

- CARDS – pachetul de cărți Șansă / Community Chest, împreună cu efectul bănesc asupra jucătorului.
- DICE_ROLLS – aruncările de zaruri făcute de un jucător, identificate prin tura în care au avut loc.
- CARD_DRAWS – evenimentul în care un jucător trage o carte din pachet în timpul partidei.

Reguli de funcționare:

- O proprietate aparține exact unui grup de culoare, iar un grup conține una sau mai multe proprietăți.
- O partidă are unul sau mai mulți jucători, iar un jucător aparține exact unei partide.
- Fiecare obiect este ori o proprietate (PROPERTY_ITEM), ori un sac de bani (CASHBAG_ITEM), niciodată ambele; coloana type stabilește subtipul.
- Deținerea curentă a unui obiect rezultă din ultima tranzacție a acelui obiect în partidă.
- O poziție de pe tablă corespunde cel mult unei proprietăți, iar o proprietate se află pe cel mult o poziție.
- Un jucător poate arunca zarurile de mai multe ori (câte o dată pe tură) și poate trage mai multe cărți pe parcursul partidei.

2. Prezentarea constrângerilor

Constrângerile modelului rezultă din cardinalitățile relațiilor și din restricțiile de domeniu.

Constrângeri de cardinalitate:

- SETS–PROPERTIES (1:N): un grup de culoare conține una sau mai multe proprietăți, iar fiecare proprietate aparține exact unui grup (properties.set_color este NOT NULL și cheie externă către sets.color).
- GAMES–PLAYERS (1:N): o partidă are unul sau mai mulți jucători, iar fiecare jucător aparține exact unei partide.
- ITEMS–PROPERTY_ITEMS și ITEMS–CASHBAG_ITEMS (1:1): fiecare rând dintr-un subtip corespunde exact unui ITEM; cheia primară a subtipului este în același timp cheie externă către items.id.
- PROPERTIES–PROPERTY_ITEMS (1:N): o proprietate poate fi reprezentată de mai multe drepturi de proprietate de-a lungul partidelor.

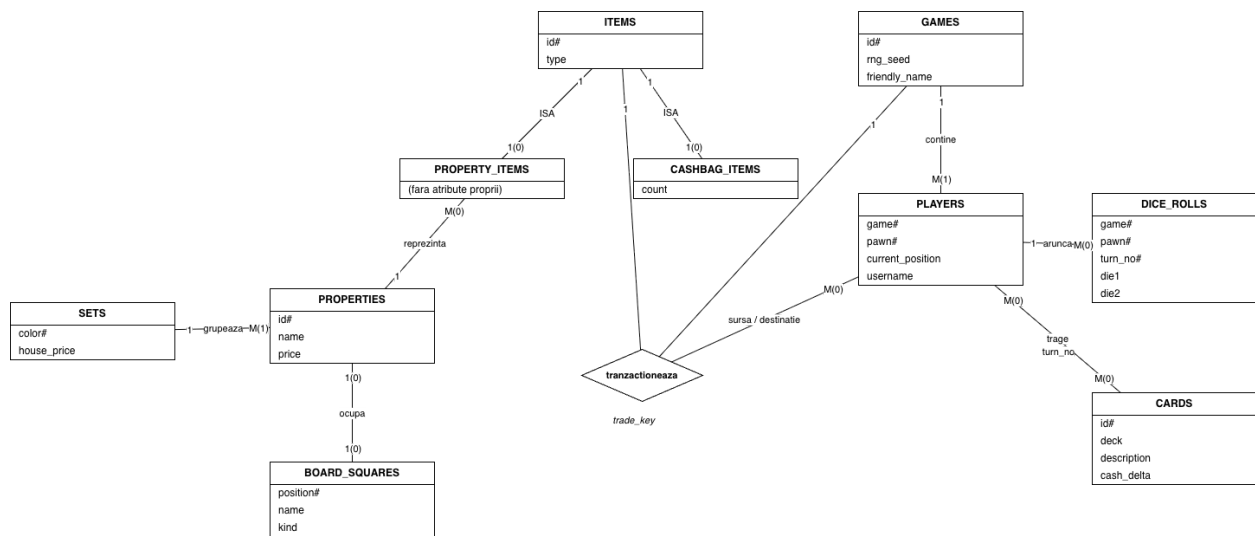
- **PROPERTIES–BOARD_SQUARES** (1:1 opțional): o proprietate se află pe cel mult o poziție, iar o poziție găzduiește cel mult o proprietate (board_squares.property este UNIQUE și poate fi NULL).
- **PLAYERS–DICE_ROLLS** (1:N) și **PLAYERS–CARD_DRAWS** (1:N): un jucător are zero sau mai multe aruncări de zaruri, respectiv trageri de cărți.
- **CARDS–CARD_DRAWS** (1:N): o carte poate fi trasă de mai multe ori.
- **PLAYERS–CARDS** (M:N): relația many-to-many este realizată prin tabelul asociativ **CARD_DRAWS**.
- Relația de tip 3 (ternară) **TRANSACTIONS** leagă simultan o partidă (**GAMES**), pionii sursă și destinație (**PLAYERS**) și obiectul mutat (**ITEMS**).

Constrângeri de domeniu și de integritate:

- Fiecare tabel are cheie primară, simplă sau compusă: **players**(game, pawn), **dice_rolls**(game, pawn, turn_no).
- **transactions.trade_key** este UNIQUE: cele două tranzacții ale unui schimb au aceeași valoare, restul sunt NULL.
- **board_squares.kind** acceptă doar valorile din lista fixă **GO, PROPERTY, RAILROAD, UTILITY, TAX, CHANCE, COMMUNITY_CHEST, JAIL, FREE_PARKING, GO_TO_JAIL** (constrângere CHECK).
- **cards.deck** poate fi doar **CHANCE** sau **COMMUNITY_CHEST** (CHECK), iar **cards.cash_delta** are valoarea implicită 0.
- **dice_rolls.die1** și **dice_rolls.die2** trebuie să fie între 1 și 6 (CHECK).
- Cheile externe compuse (game, pawn) din **TRANSACTIONS** garantează că sursa și destinația unei mutări sunt jucători ai aceleiași partide.

3. Proiectarea Diagramei Entitate-Relație

Diagrama Entitate-Relație de mai jos surprinde entitățile modelului și relațiile dintre ele. Pe baza ei se rezolvă cerințele 3.1–3.3.



3.1 Descrierea entităților

1. Entitate independentă

Entitatea independentă SETS reprezintă grupul de culoare al proprietăților. Nu depinde de existența altei entități. Cheia primară este color (numele culorii).

2. Entitate dependentă

Entitatea dependentă PLAYERS reprezintă un pion dintr-o partidă. Existența sa depinde de entitatea GAMES. Cheia primară este compusă: (game, pawn), unde game este cheie externă către GAMES, iar pawn este numărul pionului în cadrul partidei.

3. Subentitate

Subentitatea PROPERTY_ITEMS este un subtip al entității ITEMS și reprezintă deținerea unei proprietăți. Cheia primară este id, care este în același timp cheie externă către items.id (moștenire 1:1). Discriminatorul moștenirii (ISA) este coloana type din supertipul ITEMS.

3.2 Descrierea relațiilor

1. Relație one-to-one

ITEMS – PROPERTY_ITEMS: un obiect de tip proprietate corespunde exact unui PROPERTY_ITEM și invers. Cardinalitate 1:1, implementată prin faptul că cheia primară a subtipului este și cheie externă către supertip.

2. Relație one-to-many

SETS – PROPERTIES: un grup de culoare conține una sau mai multe proprietăți, iar o proprietate aparține exact unui grup. Cardinalitate 1:N.

3. Relație many-to-many

PLAYERS – CARDS (prin CARD_DRAWS): un jucător poate trage mai multe cărți, iar o carte poate fi trasă de mai mulți jucători. Cardinalitate M:N, realizată prin tabelul asociativ CARD_DRAWS.

4. Relație de tip 3

TRANSACTIONS: relație ternară care leagă simultan o partidă (GAMES), pionul sursă și pionul destinație (PLAYERS) și obiectul mutat (ITEMS) — modelând mutarea unui obiect de la un jucător la altul într-o partidă.

3.3 Descrierea atributelor

1. Atributele unei entități independente

Entitatea independentă PROPERTIES are atributele:

- id: INTEGER, cheie primară.
- set_color: VARCHAR2(255), NOT NULL, cheie externă către sets.color.
- name: VARCHAR2(255), numele proprietății.
- price: INTEGER, prețul de achiziție.

2. Atributele unei subentități

Subentitatea CASHBAG_ITEMS are doar atributele proprii (atributele moștenite din ITEMS nu se enumeră aici):

- id: INTEGER, cheie primară și, în același timp, cheie externă către items.id.
- count: INTEGER, suma de bani conținută.

Observație: coloana type nu aparține subentității, ci se moștenește din supertipul ITEMS.

3. Atributele unei relații

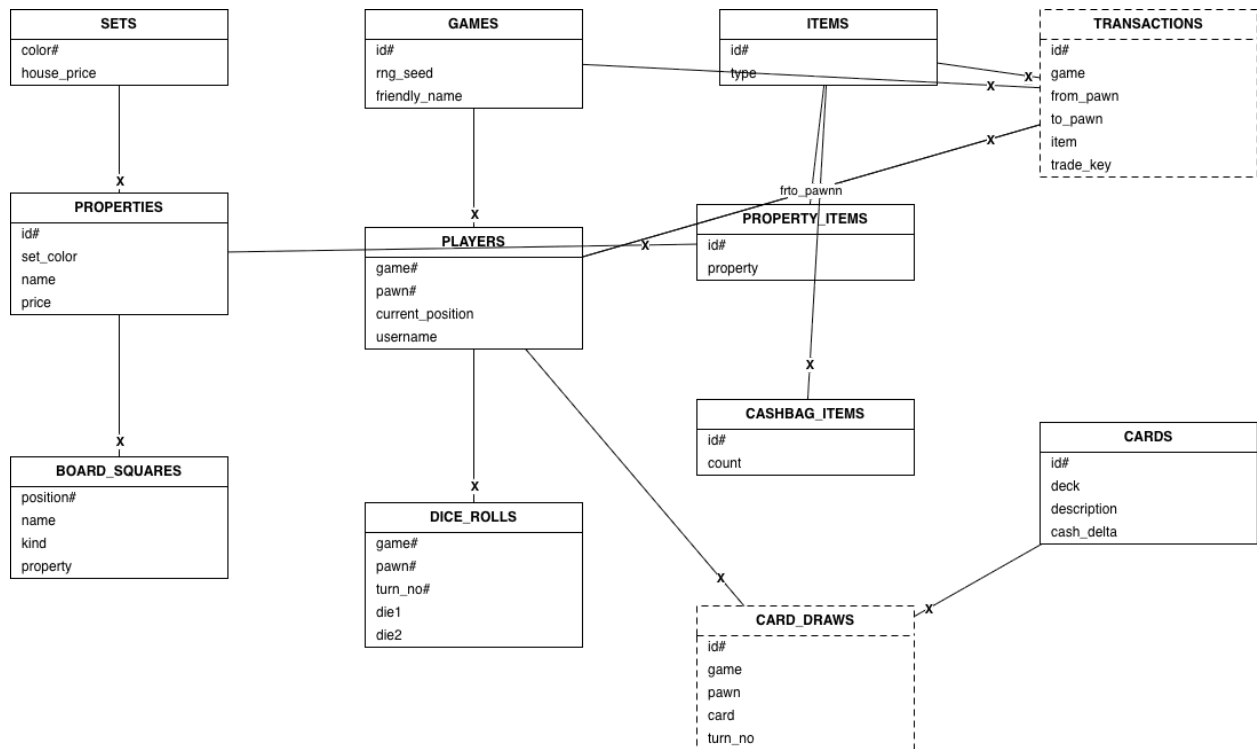
Relația TRANSACTIONS are atributele:

- id: INTEGER, cheie primară.
- game: INTEGER, cheie externă către GAMES.
- from_pawn: INTEGER, poate fi NULL (mutare de la bancă), cheie externă către PLAYERS.
- to_pawn: INTEGER, cheie externă către PLAYERS.
- item: INTEGER, cheie externă către ITEMS.
- trade_key: INTEGER, UNIQUE, implicit NULL; leagă cele două tranzacții ale unui schimb.

4. Transformarea Diagramei ER în Diagramă Conceptuală

La transformarea Diagramei Entitate-Relație în Diagrama Conceptuală, fiecare entitate devine un tabel, iar relațiile se materializează prin chei externe:

- Relația 1:N SETS-PROPERTIES se transformă prin adăugarea cheii externe set_color în tabelul PROPERTIES.
- Relația M:N dintre PLAYERS și CARDS devine tabelul asociativ CARD_DRAWS, care preia cheile primare ale ambelor entități.
- Relația ternară devine tabelul TRANSACTIONS, cu câte o cheie externă spre fiecare entitate participantă (GAMES, PLAYERS, ITEMS).
- Subtipurile PROPERTY_ITEMS și CASHBAG_ITEMS moștenesc cheia primară a supertipului ITEMS (id devine și cheie externă către ITEMS), fiind discriminate prin coloana type.
- Entitatea dependentă PLAYERS primește o cheie primară compusă (game, pawn).



5. Scheme relațională

1. Schemă relațională – tabel independent

SETS (color, house_price)

Cheie primară: color.

2. Schemă relațională – subtabel

PROPERTY_ITEMS (id, property)

Cheie primară: id. id → ITEMS(id); property → PROPERTIES(id).

3. Schemă relațională – tabel asociativ

TRANSACTIONS (id, game, from_pawn, to_pawn, item, trade_key)

Cheie primară: id. game → GAMES(id); (game, from_pawn) → PLAYERS(game, pawn);
(game, to_pawn) → PLAYERS(game, pawn); item → ITEMS(id); trade_key este UNIQUE.

6. Crearea tabelelor în SQL și inserarea datelor

6.1 Crearea tabelelor

Codul de creare a tuturor tabelelor se află în fișierul Script_Soare_Mihai_G152.txt și a fost rulat integral în Oracle. Mai jos se atașează câte un print-screen din Oracle pentru fiecare tabel creat.

```
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM Monopoly schema.
oracle-1 | SQL> REM The ERD attribute `properties.set` is mapped to the column
oracle-1 | SQL> REM `set_color` here, because `SET` is a reserved word in Oracle SQL.
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM SETS holds the color groups and the price of a house on that group.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating SETS table ....
oracle-1 | ***** Creating SETS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE sets
oracle-1 | 2      ( color          VARCHAR2(255)
oracle-1 | 3      , house_price  INTEGER
oracle-1 | 4      , CONSTRAINT sets_pk PRIMARY KEY (color)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM GAMES holds a single game of Monopoly.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating GAMES table ....
oracle-1 | ***** Creating GAMES table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE games
oracle-1 | 2      ( id              INTEGER
oracle-1 | 3      , rng_seed       INTEGER
oracle-1 | 4      , friendly_name NVARCHAR2(255)
oracle-1 | 5      , CONSTRAINT games_pk PRIMARY KEY (id)
oracle-1 | 6      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
```

```

oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM ITEMS is the supertype for anything that can be owned/traded.
oracle-1 | SQL> REM `type` discriminates between the PROPERTY_ITEMS and CASHBAG_ITEMS
oracle-1 | SQL> REM subtypes.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating ITEMS table ....
oracle-1 | ***** Creating ITEMS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE items
oracle-1 | 2      ( id          INTEGER
oracle-1 | 3      , type          INTEGER
oracle-1 | 4      , CONSTRAINT items_pk PRIMARY KEY (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM PROPERTIES holds the buyable board squares, grouped into a SET.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating PROPERTIES table ....
oracle-1 | ***** Creating PROPERTIES table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE properties
oracle-1 | 2      ( id          INTEGER
oracle-1 | 3      , set_color    VARCHAR2(255)
oracle-1 | 4      , name         VARCHAR2(255)
oracle-1 | 5      , price        INTEGER
oracle-1 | 6      , CONSTRAINT properties_pk PRIMARY KEY (id)
oracle-1 | 7      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE properties
oracle-1 | 2 ADD ( CONSTRAINT properties_set_fk
oracle-1 | 3      FOREIGN KEY (set_color)
oracle-1 | 4      REFERENCES sets (color)
oracle-1 | 5      );
oracle-1 |

```

```

oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM PLAYERS is a pawn taking part in a GAME. Identified by the game it
oracle-1 | SQL> REM belongs to plus its pawn number.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating PLAYERS table ....
oracle-1 | ***** Creating PLAYERS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE players
oracle-1 | 2      ( game          INTEGER
oracle-1 | 3      , pawn            INTEGER
oracle-1 | 4      , current_position INTEGER
oracle-1 | 5      , username        VARCHAR2(255)
oracle-1 | 6      , CONSTRAINT players_pk PRIMARY KEY (game, pawn)
oracle-1 | 7      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE players
oracle-1 | 2 ADD ( CONSTRAINT players_game_fk
oracle-1 | 3      FOREIGN KEY (game)
oracle-1 | 4      REFERENCES games (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM PROPERTY_ITEMS is a subtype of ITEMS: an item that represents the
oracle-1 | SQL> REM ownership of a PROPERTY. Its id is shared 1:1 with ITEMS.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating PROPERTY_ITEMS table ....
oracle-1 | ***** Creating PROPERTY_ITEMS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE property_items
oracle-1 | 2      ( id              INTEGER
oracle-1 | 3      , property        INTEGER
oracle-1 | 4      , CONSTRAINT property_items_pk PRIMARY KEY (id)

```

```

oracle-1 | 5      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE property_items
oracle-1 | 2 ADD ( CONSTRAINT property_items_item_fk
oracle-1 | 3      FOREIGN KEY (id)
oracle-1 | 4      REFERENCES items (id)
oracle-1 | 5      , CONSTRAINT property_items_property_fk
oracle-1 | 6      FOREIGN KEY (property)
oracle-1 | 7      REFERENCES properties (id)
oracle-1 | 8      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM CASHBAG_ITEMS is a subtype of ITEMS: an item that holds an amount
oracle-1 | SQL> REM of cash. Its id is shared 1:1 with ITEMS.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating CASHBAG_ITEMS table ....
oracle-1 | ***** Creating CASHBAG_ITEMS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE cashbag_items
oracle-1 | 2      ( id          INTEGER
oracle-1 | 3      , count        INTEGER
oracle-1 | 4      , CONSTRAINT cashbag_items_pk PRIMARY KEY (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE cashbag_items
oracle-1 | 2 ADD ( CONSTRAINT cashbag_items_item_fk
oracle-1 | 3      FOREIGN KEY (id)
oracle-1 | 4      REFERENCES items (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |

```

```

oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM TRANSACTIONS records an ITEM moving between pawns within a GAME.
oracle-1 | SQL> REM from_pawn is NULL for movements originating from the bank.
oracle-1 | SQL> REM Paired trades share a common trade_key.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating TRANSACTIONS table ....
oracle-1 | ***** Creating TRANSACTIONS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE transactions
oracle-1 | 2      ( id          INTEGER
oracle-1 | 3      , game          INTEGER
oracle-1 | 4      , from_pawn     INTEGER
oracle-1 | 5      , to_pawn       INTEGER
oracle-1 | 6      , item          INTEGER
oracle-1 | 7      , trade_key     INTEGER
oracle-1 | 8      , CONSTRAINT transactions_pk PRIMARY KEY (id)
oracle-1 | 9      , CONSTRAINT transactions_trade_key_uk UNIQUE (trade_key)
oracle-1 | 10     );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE transactions
oracle-1 | 2 ADD ( CONSTRAINT transactions_game_fk
oracle-1 | 3      FOREIGN KEY (game)
oracle-1 | 4      REFERENCES games (id)
oracle-1 | 5      , CONSTRAINT transactions_item_fk
oracle-1 | 6      FOREIGN KEY (item)
oracle-1 | 7      REFERENCES items (id)
oracle-1 | 8      , CONSTRAINT transactions_from_fk
oracle-1 | 9      FOREIGN KEY (game, from_pawn)
oracle-1 | 10     REFERENCES players (game, pawn)
oracle-1 | 11     , CONSTRAINT transactions_to_fk
oracle-1 | 12     FOREIGN KEY (game, to_pawn)
oracle-1 | 13     REFERENCES players (game, pawn)
oracle-1 | 14     );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>

```

```

oracle-1 | SQL> REM *****
oracle-1 | SQL> REM BOARD_SQUARES is the fixed board layout: one row per position on the
oracle-1 | SQL> REM board. `kind` classifies the square; `property` links the (at most
oracle-1 | SQL> REM one) PROPERTY sitting on that square, so it is UNIQUE and nullable
oracle-1 | SQL> REM (special squares such as GO or JAIL hold no property).
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating BOARD_SQUARES table ....
oracle-1 | ***** Creating BOARD_SQUARES table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE board_squares
oracle-1 | 2      ( position      INTEGER
oracle-1 | 3      , name            VARCHAR2(255)
oracle-1 | 4      , kind            VARCHAR2(20)
oracle-1 | 5      , property        INTEGER
oracle-1 | 6      , CONSTRAINT board_squares_pk PRIMARY KEY (position)
oracle-1 | 7      , CONSTRAINT board_squares_property_uk UNIQUE (property)
oracle-1 | 8      , CONSTRAINT board_squares_kind_ck CHECK
oracle-1 | 9          ( kind IN ( 'GO', 'PROPERTY', 'RAILROAD', 'UTILITY', 'TAX'
oracle-1 | 10                  , 'CHANCE', 'COMMUNITY_CHEST', 'JAIL', 'FREE_PARKING'
oracle-1 | 11                  , 'GO_TO_JAIL' ) )
oracle-1 | 12      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE board_squares
oracle-1 | 2 ADD ( CONSTRAINT board_squares_property_fk
oracle-1 | 3      FOREIGN KEY (property)
oracle-1 | 4      REFERENCES properties (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM CARDS is the Chance / Community Chest deck. `cash_delta` is the money
oracle-1 | SQL> REM the drawing player collects (positive) or pays (negative).
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating CARDS table ....
oracle-1 | ***** Creating CARDS table ....
oracle-1 | SQL>

```

```

oracle-1 | SQL> CREATE TABLE cards
oracle-1 | 2      ( id            INTEGER
oracle-1 | 3      , deck            VARCHAR2(20)
oracle-1 | 4      , description    VARCHAR2(255)
oracle-1 | 5      , cash_delta    INTEGER DEFAULT 0
oracle-1 | 6      , CONSTRAINT cards_pk PRIMARY KEY (id)
oracle-1 | 7      , CONSTRAINT cards_deck_ck CHECK (deck IN ('CHANCE', 'COMMUNITY_CHEST'))
oracle-1 | 8      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM DICE_ROLLS logs every roll a PLAYER makes, identified by the turn it
oracle-1 | SQL> REM happened on. Depends on PLAYERS, so its key includes (game, pawn).
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating DICE_ROLLS table ....
oracle-1 | ***** Creating DICE_ROLLS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE dice_rolls
oracle-1 | 2      ( game            INTEGER
oracle-1 | 3      , pawn            INTEGER
oracle-1 | 4      , turn_no        INTEGER
oracle-1 | 5      , die1            INTEGER
oracle-1 | 6      , die2            INTEGER
oracle-1 | 7      , CONSTRAINT dice_rolls_pk PRIMARY KEY (game, pawn, turn_no)
oracle-1 | 8      , CONSTRAINT dice_rolls_die1_ck CHECK (die1 BETWEEN 1 AND 6)
oracle-1 | 9      , CONSTRAINT dice_rolls_die2_ck CHECK (die2 BETWEEN 1 AND 6)
oracle-1 | 10     );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE dice_rolls
oracle-1 | 2 ADD ( CONSTRAINT dice_rolls_player_fk
oracle-1 | 3      FOREIGN KEY (game, pawn)
oracle-1 | 4      REFERENCES players (game, pawn)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |

```



```

oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM CARD_DRAWS records a PLAYER drawing a CARD during a game: the
oracle-1 | SQL> REM associative table between PLAYERS and CARDS.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating CARD_DRAWS table ....
oracle-1 | ***** Creating CARD_DRAWS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE card_draws
oracle-1 | 2      ( id          INTEGER
oracle-1 | 3      , game          INTEGER
oracle-1 | 4      , pawn          INTEGER
oracle-1 | 5      , card          INTEGER
oracle-1 | 6      , turn_no       INTEGER
oracle-1 | 7      , CONSTRAINT card_draws_pk PRIMARY KEY (id)
oracle-1 | 8      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE card_draws
oracle-1 | 2 ADD ( CONSTRAINT card_draws_player_fk
oracle-1 | 3      FOREIGN KEY (game, pawn)
oracle-1 | 4      REFERENCES players (game, pawn)
oracle-1 | 5      , CONSTRAINT card_draws_card_fk
oracle-1 | 6      FOREIGN KEY (card)
oracle-1 | 7      REFERENCES cards (id)
oracle-1 | 8      );
oracle-1 |
oracle-1 | Table altered.

```

6.2 Inserarea datelor

Comenzile de inserare (minimum 5 înregistrări în fiecare tabel) au fost rulate în Oracle. Mai jos se atașează câte un print-screen din Oracle pentru inserările din fiecare tabel.

```

oracle-1 | SQL> REM *****
oracle-1 | SQL> REM Monopoly schema.
oracle-1 | SQL> REM The ERD attribute `properties.set` is mapped to the column
oracle-1 | SQL> REM `set_color` here, because `SET` is a reserved word in Oracle SQL.

```

```

oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM SETS holds the color groups and the price of a house on that group.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating SETS table ....
oracle-1 | ***** Creating SETS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE sets
oracle-1 | 2      ( color          VARCHAR2(255)
oracle-1 | 3      , house_price   INTEGER
oracle-1 | 4      , CONSTRAINT sets_pk PRIMARY KEY (color)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM GAMES holds a single game of Monopoly.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating GAMES table ....
oracle-1 | ***** Creating GAMES table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE games
oracle-1 | 2      ( id              INTEGER
oracle-1 | 3      , rng_seed        INTEGER
oracle-1 | 4      , friendly_name  NVARCHAR2(255)
oracle-1 | 5      , CONSTRAINT games_pk PRIMARY KEY (id)
oracle-1 | 6      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM ITEMS is the supertype for anything that can be owned/traded.
oracle-1 | SQL> REM `type` discriminates between the PROPERTY_ITEMS and CASHBAG_ITEMS
oracle-1 | SQL> REM subtypes.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating ITEMS table ....
oracle-1 | ***** Creating ITEMS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE items

```

```

oracle-1 | 2      ( id          INTEGER
oracle-1 | 3      , type          INTEGER
oracle-1 | 4      , CONSTRAINT items_pk PRIMARY KEY (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM PROPERTIES holds the buyable board squares, grouped into a SET.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating PROPERTIES table ....
oracle-1 | ***** Creating PROPERTIES table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE properties
oracle-1 | 2      ( id          INTEGER
oracle-1 | 3      , set_color    VARCHAR2(255)
oracle-1 | 4      , name         VARCHAR2(255)
oracle-1 | 5      , price       INTEGER
oracle-1 | 6      , CONSTRAINT properties_pk PRIMARY KEY (id)
oracle-1 | 7      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE properties
oracle-1 | 2 ADD ( CONSTRAINT properties_set_fk
oracle-1 | 3      FOREIGN KEY (set_color)
oracle-1 | 4      REFERENCES sets (color)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM PLAYERS is a pawn taking part in a GAME. Identified by the game it
oracle-1 | SQL> REM belongs to plus its pawn number.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating PLAYERS table ....
oracle-1 | ***** Creating PLAYERS table ....
oracle-1 | SQL>

```

```

oracle-1 | SQL> CREATE TABLE players
oracle-1 | 2      ( game          INTEGER
oracle-1 | 3      , pawn            INTEGER
oracle-1 | 4      , current_position INTEGER
oracle-1 | 5      , username          VARCHAR2(255)
oracle-1 | 6      , CONSTRAINT players_pk PRIMARY KEY (game, pawn)
oracle-1 | 7      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE players
oracle-1 | 2 ADD ( CONSTRAINT players_game_fk
oracle-1 | 3      FOREIGN KEY (game)
oracle-1 | 4      REFERENCES games (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM PROPERTY_ITEMS is a subtype of ITEMS: an item that represents the
oracle-1 | SQL> REM ownership of a PROPERTY. Its id is shared 1:1 with ITEMS.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating PROPERTY_ITEMS table ....
oracle-1 | ***** Creating PROPERTY_ITEMS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE property_items
oracle-1 | 2      ( id          INTEGER
oracle-1 | 3      , property    INTEGER
oracle-1 | 4      , CONSTRAINT property_items_pk PRIMARY KEY (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE property_items
oracle-1 | 2 ADD ( CONSTRAINT property_items_item_fk
oracle-1 | 3      FOREIGN KEY (id)
oracle-1 | 4      REFERENCES items (id)
oracle-1 | 5      , CONSTRAINT property_items_property_fk

```

```

oracle-1 | 6          FOREIGN KEY (property)
oracle-1 | 7          REFERENCES properties (id)
oracle-1 | 8      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM CASHBAG_ITEMS is a subtype of ITEMS: an item that holds an amount
oracle-1 | SQL> REM of cash. Its id is shared 1:1 with ITEMS.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating CASHBAG_ITEMS table ....
oracle-1 | ***** Creating CASHBAG_ITEMS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE cashbag_items
oracle-1 | 2      ( id          INTEGER
oracle-1 | 3      , count        INTEGER
oracle-1 | 4      , CONSTRAINT cashbag_items_pk PRIMARY KEY (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE cashbag_items
oracle-1 | 2 ADD ( CONSTRAINT cashbag_items_item_fk
oracle-1 | 3      FOREIGN KEY (id)
oracle-1 | 4      REFERENCES items (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM TRANSACTIONS records an ITEM moving between pawns within a GAME.
oracle-1 | SQL> REM from_pawn is NULL for movements originating from the bank.
oracle-1 | SQL> REM Paired trades share a common trade_key.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating TRANSACTIONS table ....
oracle-1 | ***** Creating TRANSACTIONS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE transactions

```

```

oracle-1 | 2      ( id          INTEGER
oracle-1 | 3      , game          INTEGER
oracle-1 | 4      , from_pawn     INTEGER
oracle-1 | 5      , to_pawn      INTEGER
oracle-1 | 6      , item          INTEGER
oracle-1 | 7      , trade_key     INTEGER
oracle-1 | 8      , CONSTRAINT transactions_pk PRIMARY KEY (id)
oracle-1 | 9      , CONSTRAINT transactions_trade_key_uk UNIQUE (trade_key)
oracle-1 | 10     );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE transactions
oracle-1 | 2 ADD ( CONSTRAINT transactions_game_fk
oracle-1 | 3      FOREIGN KEY (game)
oracle-1 | 4      REFERENCES games (id)
oracle-1 | 5      , CONSTRAINT transactions_item_fk
oracle-1 | 6      FOREIGN KEY (item)
oracle-1 | 7      REFERENCES items (id)
oracle-1 | 8      , CONSTRAINT transactions_from_fk
oracle-1 | 9      FOREIGN KEY (game, from_pawn)
oracle-1 | 10     REFERENCES players (game, pawn)
oracle-1 | 11     , CONSTRAINT transactions_to_fk
oracle-1 | 12     FOREIGN KEY (game, to_pawn)
oracle-1 | 13     REFERENCES players (game, pawn)
oracle-1 | 14     );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM BOARD_SQUARES is the fixed board layout: one row per position on the
oracle-1 | SQL> REM board. `kind` classifies the square; `property` links the (at most
oracle-1 | SQL> REM one) PROPERTY sitting on that square, so it is UNIQUE and nullable
oracle-1 | SQL> REM (special squares such as GO or JAIL hold no property).
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating BOARD_SQUARES table ....
oracle-1 | ***** Creating BOARD_SQUARES table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE board_squares

```

```

oracle-1 | 2      ( position      INTEGER
oracle-1 | 3      , name          VARCHAR2(255)
oracle-1 | 4      , kind          VARCHAR2(20)
oracle-1 | 5      , property      INTEGER
oracle-1 | 6      , CONSTRAINT board_squares_pk PRIMARY KEY (position)
oracle-1 | 7      , CONSTRAINT board_squares_property_uk UNIQUE (property)
oracle-1 | 8      , CONSTRAINT board_squares_kind_ck CHECK
oracle-1 | 9          ( kind IN ( 'GO', 'PROPERTY', 'RAILROAD', 'UTILITY', 'TAX'
oracle-1 | 10              , 'CHANCE', 'COMMUNITY_CHEST', 'JAIL', 'FREE_PARKING'
oracle-1 | 11              , 'GO_TO_JAIL' ) )
oracle-1 | 12      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE board_squares
oracle-1 | 2 ADD ( CONSTRAINT board_squares_property_fk
oracle-1 | 3      FOREIGN KEY (property)
oracle-1 | 4      REFERENCES properties (id)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM CARDS is the Chance / Community Chest deck. `cash_delta` is the money
oracle-1 | SQL> REM the drawing player collects (positive) or pays (negative).
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating CARDS table ....
oracle-1 | ***** Creating CARDS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE cards
oracle-1 | 2      ( id              INTEGER
oracle-1 | 3      , deck            VARCHAR2(20)
oracle-1 | 4      , description     VARCHAR2(255)
oracle-1 | 5      , cash_delta      INTEGER DEFAULT 0
oracle-1 | 6      , CONSTRAINT cards_pk PRIMARY KEY (id)
oracle-1 | 7      , CONSTRAINT cards_deck_ck CHECK (deck IN ('CHANCE', 'COMMUNITY_CHEST'))
oracle-1 | 8      );
oracle-1 |
oracle-1 | Table created.

```

```

oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM DICE_ROLLS logs every roll a PLAYER makes, identified by the turn it
oracle-1 | SQL> REM happened on. Depends on PLAYERS, so its key includes (game, pawn).
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating DICE_ROLLS table ....
oracle-1 | ***** Creating DICE_ROLLS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE dice_rolls
oracle-1 | 2      ( game          INTEGER
oracle-1 | 3      , pawn            INTEGER
oracle-1 | 4      , turn_no        INTEGER
oracle-1 | 5      , die1            INTEGER
oracle-1 | 6      , die2            INTEGER
oracle-1 | 7      , CONSTRAINT dice_rolls_pk PRIMARY KEY (game, pawn, turn_no)
oracle-1 | 8      , CONSTRAINT dice_rolls_die1_ck CHECK (die1 BETWEEN 1 AND 6)
oracle-1 | 9      , CONSTRAINT dice_rolls_die2_ck CHECK (die2 BETWEEN 1 AND 6)
oracle-1 | 10     );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE dice_rolls
oracle-1 | 2 ADD ( CONSTRAINT dice_rolls_player_fk
oracle-1 | 3      FOREIGN KEY (game, pawn)
oracle-1 | 4      REFERENCES players (game, pawn)
oracle-1 | 5      );
oracle-1 |
oracle-1 | Table altered.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> REM *****
oracle-1 | SQL> REM CARD_DRAWS records a PLAYER drawing a CARD during a game: the
oracle-1 | SQL> REM associative table between PLAYERS and CARDS.
oracle-1 | SQL>
oracle-1 | SQL> Prompt ***** Creating CARD_DRAWS table ....
oracle-1 | ***** Creating CARD_DRAWS table ....
oracle-1 | SQL>
oracle-1 | SQL> CREATE TABLE card_draws
oracle-1 | 2      ( id              INTEGER

```



```

oracle-1 | 3      , game          INTEGER
oracle-1 | 4      , pawn            INTEGER
oracle-1 | 5      , card             INTEGER
oracle-1 | 6      , turn_no        INTEGER
oracle-1 | 7      , CONSTRAINT card_draws_pk PRIMARY KEY (id)
oracle-1 | 8      );
oracle-1 |
oracle-1 | Table created.
oracle-1 |
oracle-1 | SQL>
oracle-1 | SQL> ALTER TABLE card_draws
oracle-1 | 2  ADD ( CONSTRAINT card_draws_player_fk
oracle-1 | 3          FOREIGN KEY (game, pawn)
oracle-1 | 4          REFERENCES players (game, pawn)
oracle-1 | 5      , CONSTRAINT card_draws_card_fk
oracle-1 | 6          FOREIGN KEY (card)
oracle-1 | 7          REFERENCES cards (id)
oracle-1 | 8      );
oracle-1 |
oracle-1 | Table altered.

```